

Capstone Proposal — Praxis

An agent that distills its own session logs into durable, reusable knowledge.

One line: Claude Code's auto-memory already saves a few notes between sessions — but it's an unverified black box (no human approval, no dedup, no measurement). Praxis mines the *full* JSONL logs the agent produces, distills the real lessons, runs them through a confidence + human-approval gate, and feeds the promoted ones back into future sessions — so the agent provably stops relearning the same things and gets better over time.

Problem

Coding agents used to be amnesiac; they're less so now. Claude Code's **auto-memory** (GA, v2.1.59+) lets the model decide what's worth remembering as it works — build commands, debugging insights, style preferences — and write it as markdown under `~/.claude/projects/<project>/memory/`, auto-loaded next session. That's a real step. But it's a thin, best-effort layer, and its gaps are exactly where durable knowledge lives:

- **No quality gate.** The save decision is an opaque, model-driven judgment with *no human approval* — it can memorize a wrong pattern from a one-off mistake as readily as a real lesson, and you only learn what it saved by running `/memory`.
- **No dedup, decay, or conflict resolution.** Stale and contradictory notes coexist indefinitely; nothing ages out or reconciles them.
- **No measurement.** Nothing verifies a saved memory actually helped a later session.
- **Per-repository only.** Nothing carries across projects, models, or domains — and not everyone using Claude Code is even coding.
- **Best-effort recall.** Startup loads the first ~25KB of `MEMORY.md` with no relevance ranking, and compliance isn't guaranteed.

Meanwhile the richer raw material is right there and underused. Claude Code persists every session as a full JSONL transcript (`~/.claude/projects/<project>/<session>.jsonl`; confirmed here — ~20 projects) — user text, assistant thinking/text, tool_use, tool_result: a near-complete record of every mistake, correction, dead end, and success. Auto-memory skims this in-flight and discards the rest; nobody mines the *complete* corpus offline. **It's exhaust today; it should be a compounding asset.**

The framing: **memory vs. knowledge.** Auto-memory captures scattered *memory* — episodic, unscored, unverified. We want *knowledge* — generalized, deduped, confidence-scored, human-approved, measured, and portable across projects and domains. Praxis is the disciplined distillation layer that auto-memory gestures at but doesn't deliver.

Direction

A self-improving knowledge loop built on logs the agent already produces:

raw logs → extract candidate lessons → consolidate/dedup/generalize → confidence score → [human approval gate] → promoted knowledge → injected into future sessions → measure improvement → repeat

Commitments: **automatic** (mined, not hand-written); **promotion lifecycle with a human gate** (proposed → suggested → active → decayed); **provenance** (every item links to the exact log line that produced it); **measured** (an eval harness proves compounding rather than asserting it).

This hits hard on three different axis: **(A) ML+LLM** — classical ML clusters/dedups/scores and classifies learning moments, LLMs distill and generalize; **(B) different take on agents** — the *substrate* improves while agents stay stateless (compound engineering); **(C) many approaches** — “where does knowledge live and how is it retrieved?” benchmarked across markdown/skills vs. vector RAG vs. knowledge graph vs. RAG+rerank.

Preliminary Technical Approach

1. **Ingest & segment** JSONL into *episodes* (problem-solving arcs): heuristic boundaries → LLM-refined, vs. embedding-based segmentation.
2. **Detect learning moments** — user corrections, failure→fix transitions, explicit preferences, cross-session rediscovery: heuristics vs. a *trained classifier* (the ML hook) vs. LLM tagging.
3. **Distill** each into a structured candidate {when-it-applies, lesson, scope, evidence, form}: *extractive* vs. *abstractive* is a first-class experiment (research favors extractive).
4. **Consolidate, score, gate** — cluster/dedup (embeddings + HDBSCAN), generalize, **detect contradictions** for human resolution, **confidence score** (evidence frequency, recency, breadth; stretch: outcome correlation) with **decay** for stale lessons, behind a **human approval dashboard**.
5. **Inject & retrieve** without context bloat (lost-in-the-middle is real): generated `CLAUDE.md/skills` with just-in-time disclosure, benchmarked against vector RAG / KG / RAG+rerank.

Spine — eval harness: fixed tasks on a fixed (deliberately quirky) repo, run cold vs. with injected knowledge. Metrics: corrections, repeated failures, tokens, time, success.

Scope

MVP: ingest + segment real logs → learning-moment detection (heuristics + LLM) → LLM distillation with provenance → simple cluster/dedup + confidence → **review dashboard with the proposed→suggested→active gate** → injection via generated `CLAUDE.md/skills` → **eval harness** measuring correction-rate before/after.

Stretch: trained classifier; substrate bake-off; confidence decay + re-verification; auto outcome-correlation; contradiction-resolution UI; auto-generated hooks; cross-project/global knowledge. **Out of scope:** training from scratch; hosted SaaS; non-Claude-Code agents; real-time mid-session learning (we distill *between* sessions).

Target Outcome

Point Praxis at a repo's logs → a ranked list of candidate lessons with evidence in minutes → a human promotes the good ones in a few clicks → a re-run with promoted knowledge injected shows a **quantified drop** in corrections/failures/tokens vs. cold, plus a **compounding curve** of correction-rate falling across sessions. **Success criterion:** ≥50% fewer user corrections than cold runs on our benchmark, with no regression in success rate.

Why It's Compelling

- **Universal, real pain on a feature that already exists** — auto-memory shipping GA proves the demand; its black-box, unverified, per-repo design is precisely what we make disciplined and measurable.
 - **Turns waste into a flywheel** — discarded logs become the thing that makes the agent better the more you use it.
 - **"Agents that learn from their own mistakes"** is a sticky, true story — and we *show the curve*, not just claim it.
 - **Generalizes beyond code.** We target coding sessions because that's the data we have and the quirks demo cleanly — but nothing in the pipeline is code-specific. A Claude Code user isn't necessarily *coding*: research, writing, data, and ops workflows produce the same logs with the same signals. The loop is domain-agnostic; only the injected *form* (lint rule vs. style guideline vs. research convention) is domain-flavored — so the bet is far bigger than "a tool for coders."
 - **Grounded in current research** (context engineering, extractive-vs-abstractive, context rot, parametric-vs-contextual conflict) and **genuinely hard, not polish-bait**.
-

Hard / Risky

- **Negative compounding (central risk):** one wrong/over-generalized lesson degrades *every* future session. The human gate, provenance, thresholds, and decay all exist to prevent poisoning the well.
 - **Signal vs. noise** — logs are mostly mundane; isolating durable lessons is the hardest extraction problem. **Generalization vs. overfitting** — getting *scope* right is subtle.
 - **Honest measurement** — non-determinism and a drifting codebase make a fair before/after eval real work, and it's where credibility lives.
 - **Context budget** — even good lessons hurt if all dumped in; retrieval/prioritization is mandatory. **Privacy** — scrub secrets/PII before any lesson leaves the machine.
-

Demo On Stage (~3 live acts)

1. **Dumb agent:** fresh repo with a couple of deliberate quirks; give the agent a task — it stumbles, gets corrected, retries. Capture the log.
2. **Distillation:** run Praxis on that log; the dashboard fills with candidate lessons, each scored and linked to the exact transcript line. One click promotes `suggested → active`. (Bonus: surface and resolve a contradiction between two past sessions.)
3. **Smart agent:** re-run a sibling task — nails the quirks first try. Side-by-side scoreboard (before: 5 corrections / 12 min vs. after: 0 / 3 min), then the **compounding curve** across a pre-run batch to prove it's a trend.

The beat: the agent made a mistake once and never makes it again — automatically. We show it, not say it.